ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Decision Hub

Централизованный движок бизнес-решений для банковских операций

Документ:	Спецификация требований к программному обеспечению (СТПО)
Версия:	1.0
Статус:	Финальный
Дата:	2026-03-26
Аудитория:	Разработчики, системные аналитики, архитекторы
Кодовое имя:	decision-hub-sandbox

Содержание

1. **Введение и назначение системы**
   - 1.1 Контекст и проблематика
   - 1.2 Цели проекта
   - 1.3 Термины и определения
2. **Архитектура системы**
   - 2.1 Высокоуровневая схема
   - 2.2 Граница ответственности сервисов
   - 2.3 Принципы проектирования
3. **Требования к базе данных**
   - 3.1 Схема таблиц: payment-service
   - 3.2 Схема таблиц: decision-hub
   - 3.3 Схема таблиц: ledger-mock
4. **Требования к сервисам**
   - 4.1 api-gateway (:8000)
   - 4.2 payment-service (:8001)
   - 4.3 decision-hub (:8002)
   - 4.4 ledger-mock (:8003)
5. **API-контракты**
   - 5.1 POST /api/p2p/transfer
   - 5.2 POST /api/p2p/transfer-legacy
   - 5.3 POST /decision/evaluate
   - 5.4 GET /decision/rules + PATCH /decision/rules/{rule_id}
   - 5.5 GET /decision/audit/{decision_id}
   - 5.6 POST /ledger/posting
6. **Логика принятия решений**
   - 6.1 Правила в seed-данных
   - 6.2 Стратегия вычисления
   - 6.3 Типы условий
7. **Нефункциональные требования**
   - 7.1 Идемпотентность
   - 7.2 Структурированное логирование
   - 7.3 Propagation X-Correlation-Id
8. **Статус-машина перевода**
9. **Shared-библиотека**
10. **Конфигурация и деплой**

10.1 Переменные окружения
10.2 Docker Compose
10.3 Alembic-миграции
11. Структура репозитория
12. Демо-сценарии (4 шт.)
13. Антипаттерн AS-IS vs TO-BE
14. Критерии приёмки

1. Введение и назначение системы

1.1 Контекст и проблематика

В большинстве банков бизнес-логика принятия решений (лимиты, AML-проверки, антифрод) рассеяна по десяткам сервисов в виде хардкодированных условий: `if amount > 10_000_000: reject`. Это приводит к системным проблемам:

- Нет объяснимости: невозможно ответить на вопрос «какое именно правило сработало?»
- Нет владельца: правило AML живёт в коде инженера — не в компетенции комплаенс-отдела
- Нет версионирования: изменение порога — это коммит, PR, деплой, CAB-согласование
- Дублирование: каждый сервис реализует одни и те же проверки независимо
- Нет аудита: решение принято, но записи нет — регулятор спросит, ответить нечем
- Тестирование сложно: правило нельзя проверить без поднятия всего сервиса

Настоящая спецификация описывает архитектуру **Decision Hub** — централизованного движка решений, который решает все перечисленные проблемы. Система реализуется как production-like sandbox для обучения системных аналитиков думать на уровне инженеров.

1.2 Цели проекта

#	Цель	Метрика успеха
1	Вынести решения из кода сервисов в данные	PATCH правила без деплоя — за < 1 сек
2	Обеспечить объяснимость каждого решения	Каждый reject содержит rule_id + owner + reason_code
3	Создать неизменяемый аудит-трейл	100% решений записаны в decision_audit
4	Исключить дублирование проводки (идемпотентность)	Повтор запроса с тем же ключом → один posting
5	Явная статус-машина перевода	Все переходы логируются с correlation_id
6	Сравнить AS-IS и TO-BE архитектуру	endpoint /p2p/transfer-legacy демонстрирует антипаттерн

1.3 Термины и определения

Термин	Определение
Decision Hub	Сервис, владеющий правилами и их вычислением. Не знает о переводах.
Rule (правило)	Запись в БД: условие + действие + владелец. Меняется без деплоя.
REJECT	Решение, блокирующее операцию. Первый REJECT останавливает обработку.
CHALLENGE	Мягкий флаг: операция разрешена, но требует ручной проверки. Накапливается.

Термин	Определение
APPROVE	Решение без ограничений: все правила проверены, ни одно не сработало.
Idempotency-Key	Клиентский UUID в заголовке. Гарантирует, что повтор не создаст второй перевод.
X-Correlation-Id	UUID, сгенерированный api-gateway. Сквозной ID для трассировки через все сервисы.
decision_audit	Неизменяемая таблица. Каждое вычисление пишет snapshot контекста и результат.
Ledger Posting	Факт проводки денег. DECIDED ≠ POSTED: они независимые события.
AS-IS	Антипаттерн: решения внутри сервиса в виде if-else. Намеренно реализован для сравнения.
TO-BE	Целевая архитектура: сервис делегирует решение Decision Hub.

2. Архитектура системы

2.1 Высокоуровневая схема

Система состоит из 4 сервисов и одного экземпляра PostgreSQL 16:

```
Client (curl / mobile app)
       |
       ▼
api-gateway     :8000   —— X-Correlation-Id injection, routing, NO business logic
       |
       ▼
payment-service :8001   —— transfer lifecycle, idempotency, status machine
       |                       |
       ▼                       ▼
decision-hub    :8002   ledger-mock  :8003
rules as data,          posting simulation,
audit trail,            fail-on-demand
explainability

———————————— PostgreSQL 16 ——————————————————
payment_transfers   payment_idempotency
decision_rules      decision_audit
ledger_postings
```

2.2 Граница ответственности сервисов

Сервис	Владеет	НЕ владеет
api-gateway	Context propagation, routing, rate-limit headers	Любая бизнес-логика
payment-service	Сущность Transfer, переходы статусов, идемпотентность	Логика решений
decision-hub	Правила, движок вычисления, аудит-трейл	Состояние перевода
ledger-mock	Симуляция проводки денег	Решения, статус перевода

2.3 Принципы проектирования

- **API-first:** OpenAPI-контракты написаны до кода и являются единственным источником правды об интерфейсе.
- **Ownership over integration:** сервис либо владеет данными, либо запрашивает решение — никогда оба сразу.
- **Rules as data:** бизнес-правила хранятся в базе данных, не в коде. Изменение — PATCH-запрос, не деплой.
- **Explicit state machine:** все переходы статусов перевода явные, логируются, не имеют неявных путей.

- **Immutable audit:** каждое решение записывается в decision_audit с полным snapshot контекста. Запись не обновляется.
- **Idempotency by design:** повторный запрос с тем же Idempotency-Key возвращает кэшированный ответ — клиент не может отличить retry от оригинала.
- **Structured logging everywhere:** каждая строка лога — JSON-объект с обязательными полями service, correlation_id, event.

3. Требования к базе данных

Все сервисы используют один инстанс PostgreSQL 16. Схема разделена по префиксам таблиц, чтобы явно показать владение данными. Миграции управляются Alembic (per-service, каждый сервис мигрирует свои таблицы при старте).

3.1 Таблицы payment-service

payment_transfers

Колонка	Тип	Ограничение	Описание
id	UUID	PK	Идентификатор перевода
idempotency_key	VARCHAR	UNIQUE NOT NULL	Клиентский ключ идемпотентности
client_id	VARCHAR	NOT NULL	Идентификатор отправителя
receiver_id	VARCHAR	NOT NULL	Идентификатор получателя
amount	NUMERIC(18,2)	NOT NULL	Сумма перевода
currency	VARCHAR(3)	NOT NULL	Валюта (ISO 4217: KZT, USD...)
country	VARCHAR(2)	NOT NULL	Страна назначения (ISO 3166-1 alpha-2)
device_trust	VARCHAR	NOT NULL	Уровень доверия устройству: HIGH | MEDIUM | LOW
daily_sum	NUMERIC(18,2)	NOT NULL	Сумма переводов за сегодня до текущего
status	VARCHAR	NOT NULL	NEW | DECIDED | POSTED | REJECTED | FAILED
decision_id	UUID	NULLABLE	Ссылка на decision_audit.decision_id
correlation_id	VARCHAR	NULLABLE	X-Correlation-Id от api-gateway
created_at	TIMESTAMPTZ	NOT NULL	Время создания записи
updated_at	TIMESTAMPTZ	NOT NULL	Время последнего обновления

payment_idempotency

Колонка	Тип	Ограничение	Описание
idempotency_key	VARCHAR	PK	Тот же ключ, что в заголовке Idempotency-Key
transfer_id	UUID	NOT NULL	Ссылка на payment_transfers.id

Колонка	Тип	Ограничение	Описание
response_snapshot	JSONB	NOT NULL	Полный HTTP-ответ первого запроса (кэш для replay)
created_at	TIMESTAMPTZ	NOT NULL	Время первого запроса

Зачем отдельная таблица идемпотентности? Хранение полного response_snapshot позволяет вернуть клиенту ровно тот же ответ при повторе — включая transfer_id, статус, decision-данные. Уникальный ключ по idempotency_key исключает race condition при параллельных запросах.

3.2 Таблицы decision-hub

decision_rules

Колонка	Тип	Ограничение	Описание
rule_id	VARCHAR	PK	Человекочитаемый идентификатор: LIMIT_DAILY, AML_102...
version	VARCHAR	NOT NULL	Семантическая версия правила: 1.0, 2.1...
priority	INTEGER	NOT NULL	Порядок вычисления (меньше = раньше). 1 = высший.
active	BOOLEAN	NOT NULL	Активно ли правило. False = исключено из вычисления.
condition_type	VARCHAR	NOT NULL	THRESHOLD | BLOCKLIST | COMPOSITE
condition_params	JSONB	NOT NULL	Параметры условия (зависят от condition_type)
action	VARCHAR	NOT NULL	Что делать при срабатывании: REJECT | CHALLENGE | APPROVE
reason_code	VARCHAR	NOT NULL	Машиночитаемый код причины: AML_COUNTRY_BLOCKED
owner	VARCHAR	NOT NULL	Команда-владелец: compliance | risk | fincontrol
updated_at	TIMESTAMPTZ	NOT NULL	Время последнего изменения правила

decision_audit

Колонка	Тип	Ограничение	Описание
id	UUID	PK	Внутренний идентификатор записи
decision_id	UUID	NOT NULL, INDEX	Внешний идентификатор решения. По нему делают GET /audit/{id}
transfer_context	JSONB	NOT NULL	Полный snapshot входных данных (amount, country, device_trust...)

Колонка	Тип	Ограничение	Описание
rules_checked	JSONB	NOT NULL	Список [{rule_id, matched, action, priority}] всех проверенных правил
rules_matched	JSONB	NOT NULL	Только сработавшие правила [{rule_id, reason_code, owner, action}]
final_decision	VARCHAR	NOT NULL	APPROVE | REJECT | CHALLENGE
risk_score	NUMERIC(4,2)	NULLABLE	Числовой сигнал риска 0.0–1.0. NULL = чистый APPROVE.
correlation_id	VARCHAR	NULLABLE	X-Correlation-Id для трассировки
created_at	TIMESTAMPTZ	NOT NULL	Время создания записи. НЕИЗМЕНЯЕМО.

 Таблица decision_audit — ТОЛЬКО на запись. Записи не обновляются и не удаляются. Она является аудиторским журналом: каждое вычисление правил должно создавать новую запись.

3.3 Таблицы ledger-mock

ledger_postings

Колонка	Тип	Ограничение	Описание
id	UUID	PK	Идентификатор проводки
transfer_id	UUID	UNIQUE NOT NULL	Ссылка на перевод. UNIQUE — идемпотентность на уровне ledger.
amount	NUMERIC(18,2)	NOT NULL	Сумма проводки
currency	VARCHAR(3)	NOT NULL	Валюта
status	VARCHAR	NOT NULL	POSTED | FAILED
correlation_id	VARCHAR	NULLABLE	X-Correlation-Id для трассировки
created_at	TIMESTAMPTZ	NOT NULL	Время создания проводки

UNIQUE на transfer_id — ключевое ограничение. Если payment-service вызывает ledger дважды с тем же transfer_id (сбой сети + retry), второй вызов возвращает существующую проводку вместо создания новой. Это защита от двойного списания на уровне БД.

4. Требования к сервисам

4.1 api-gateway (:8000)

Ответственность: принимает все внешние запросы, инжектирует X-Correlation-Id, проксирует на payment-service. Не содержит бизнес-логики.

Требования к реализации:

- Реализован на FastAPI + Uvicorn, Python 3.12
- Для каждого входящего запроса: если X-Correlation-Id отсутствует — генерировать UUID4 и добавить в заголовок
- Добавлять в ответ заголовки: X-Correlation-Id, X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset
- Проксировать POST /api/p2p/transfer → payment-service:8001/p2p/transfer
- Проксировать POST /api/p2p/transfer-legacy → payment-service:8001/p2p/transfer-legacy
- Пробрасывать заголовки: Idempotency-Key, Content-Type, X-Fail-Mode, X-Correlation-Id
- GET /health: возвращать {"status": "ok", "service": "api-gateway"}
- При ошибке downstream: пробрасывать статус-код и тело ошибки без изменений
- Не хранить состояние. Не подключаться к БД.

Среда:

```
PAYMENT_SERVICE_URL=http://payment-service:8001
```

4.2 payment-service (:8001)

Ответственность: создаёт и управляет сущностью Transfer. Делегирует решение в decision-hub. Делегирует проводку в ledger-mock. Управляет идемпотентностью. Не содержит логики принятия решений.

Маршруты:

Метод	Путь	Описание
POST	/p2p/transfer	TO-BE: 8-шаговый flow через Decision Hub
POST	/p2p/transfer-legacy	AS-IS: антипаттерн с inline if-else (для сравнения)
GET	/p2p/transfers/{id}	Получить текущее состояние перевода по UUID
GET	/health	Health check с проверкой подключения к БД

TO-BE flow (POST /p2p/transfer) — 8 шагов:

- 1. Проверить наличие Idempotency-Key в заголовке → 400 IDEMPOTENCY_KEY_MISSING если отсутствует
- 2. Проверить таблицу payment_idempotency → если ключ найден, вернуть response_snapshot (HTTP 200, idempotent: true)

- 3. Создать запись PaymentTransfer со статусом NEW, сохранить в БД
- 4. Вызвать decision-hub POST /decision/evaluate → при сетевой ошибке: NEW→FAILED, сохранить идемпотентность, HTTP 503
- 5. Если decision=REJECT: NEW→REJECTED, сохранить идемпотентность, вернуть ответ со структурированными причинами
- 6. Если decision=APPROVE|CHALLENGE: NEW→DECIDED
- 7. Вызвать ledger-mock POST /ledger/posting → при ошибке: DECIDED→FAILED
- 8. При успехе: DECIDED→POSTED, сохранить идемпотентность, вернуть финальный ответ

Требования:

- Каждый переход статуса логировать как JSON: {event: status_transition, from_status, to_status, transfer_id, correlation_id}
- CorrelationMiddleware с generate_if_missing=False (gateway генерирует, сервис только читает)
- Timeout при обращении к decision-hub: 10 секунд
- Timeout при обращении к ledger-mock: 12 секунд
- Идемпотентность должна работать для всех конечных состояний: REJECTED, FAILED, POSTED

4.3 decision-hub (:8002)

Ответственность: хранит правила, вычисляет их против переданного контекста, записывает аудит-трейл. Не знает о сущности Transfer.

Маршруты:

Метод	Путь	Описание
POST	/decision/evaluate	Запустить движок правил. Записать аудит.
GET	/decision/rules	Список всех правил с полными параметрами
PATCH	/decision/rules/{rule_id}	Обновить active и/или condition_params. Без деплоя.
GET	/decision/audit/{decision_id}	Получить полную запись аудита по decision_id
GET	/health	Health check с проверкой подключения к БД

Требования к движку:

- Правила загружаются из БД при каждом вызове /decision/evaluate (не кешируются в памяти — изменения сразу видны)
- Правила сортируются по полю priority ASC перед вычислением
- Вычисляются только правила с active=True
- Первый REJECT останавливает вычисление (early exit)
- CHALLENGE-правила накапливаются и не останавливают вычисление
- Аудит-запись создаётся для КАЖДОГО вызова, включая APPROVE
- decision_id генерируется в evaluate, записывается в аудит, возвращается клиенту

4.4 ledger-mock (:8003)

Ответственность: симулирует факт движения денег. Реализует идемпотентность на уровне transfer_id. Поддерживает управляемые режимы сбоя для тестирования.

Маршруты:

Метод	Путь	Описание
POST	/ledger/posting	Создать проводку. Идемпотентно по transfer_id.
GET	/health	Health check

Управляемые сбои (X-Fail-Mode):

Значение	Поведение
(отсутствует)	Нормальная работа: создать posting, вернуть 200
TIMEOUT	asyncio.sleep(5 сек), затем успешный ответ — имитация медленного ledger
ERROR	Немедленный HTTP 500 Internal Server Error
DUPLICATE_TEST	Вернуть существующий posting если transfer_id уже есть (тест идемпотентности)

5. API-контракты

Полные OpenAPI 3.1 спецификации хранятся в каталоге `contracts/`. В этом разделе — детальное описание каждого эндпоинта в формате ТЗ.

5.1 POST /api/p2p/transfer (TO-BE)

Заголовки запроса:

Заголовок	Обязателен	Описание
Content-Type	Да	application/json
Idempotency-Key	Да	UUID4, генерируется клиентом. При отсутствии — HTTP 400.
X-Correlation-Id	Нет	Если не передан, api-gateway генерирует и добавляет.
X-Fail-Mode	Нет	Проброс в ledger: TIMEOUT | ERROR | DUPLICATE_TEST

Тело запроса:

Поле	Тип	Обязателен	Описание
client_id	string	Да	Идентификатор клиента-отправителя
receiver_id	string	Да	Идентификатор получателя
amount	float	Да	Сумма перевода (> 0)
currency	string	Да	ISO 4217: KZT, USD, EUR...
country	string	Да	ISO 3166-1 alpha-2: KZ, RU, IR...
device_trust	string	Да	HIGH | MEDIUM | LOW
daily_sum	float	Да	Накопленная сумма переводов клиента за сегодня

Примеры ответов:

HTTP 200 — перевод выполнен (POSTED):

```
{
  "transfer_id": "uuid-...",
  "status": "POSTED",
  "idempotent": false,
  "decision": {
    "decision_id": "uuid-...",
    "allowed": true,
    "decision": "APPROVE",
    "reasons": [],
    "risk_score": null,
    "rules_evaluated": 3,
    "rules_matched": 0
  },
  "posting": {"posting_id": "uuid-...", "status": "POSTED"},
```

```
  "correlation_id": "corr-xyz"
}
```

HTTP 200 — отклонён (REJECTED):

```
{
  "transfer_id": "uuid-...",
  "status": "REJECTED",
  "idempotent": false,
  "decision": {
    "decision_id": "uuid-...",
    "allowed": false,
    "decision": "REJECT",
    "reasons": [{
      "rule_id": "AML_102",
      "reason_code": "AML_COUNTRY_BLOCKED",
      "owner": "compliance"
    }],
    "risk_score": 0.95,
    "rules_evaluated": 2,
    "rules_matched": 1
  },
  "correlation_id": "corr-xyz"
}
```

HTTP 400 — отсутствует Idempotency-Key:

```
{"error_code": "IDEMPOTENCY_KEY_MISSING", "message": "...", "correlation_id": "...", "se
rvice": "payment-service"}
```

HTTP 503 — decision-hub недоступен:

```
{"error_code": "DECISION_HUB_UNAVAILABLE", "transfer_id": "uuid-...", "status": "FAILED"
, ...}
```

5.2 POST /api/p2p/transfer-legacy (AS-IS)

Антипаттерн-эндпоинт. Та же сигнатура запроса, что и /transfer. Принципиальное отличие в ответе:

Аспект	AS-IS (legacy)	TO-BE (Decision Hub)
Отклонение	{"status": "REJECTED", "reason": "AML_BLOCKED"}	{"status": "REJECTED", "decision": {"reasons": [{"rule_id": "AML_102", "reason_code": "AML_COUNTRY_BLOCKED", "owner": "compliance"}], ...}}
Аудит	Нет записи	Запись в decision_audit
rule_id	Нет	Есть
owner	Нет	Есть
version	Нет	Есть (в decision_rules.version)
Изменение правила	Код + деплой	PATCH /decision/rules/{id}

5.3 POST /decision/evaluate

Запрос:

```
{
  "decision_type": "P2P_TRANSFER",
  "correlation_id": "corr-xyz",
  "context": {
    "client_id": "...", "receiver_id": "...",
    "amount": 50000, "currency": "KZT",
    "country": "IR", "device_trust": "HIGH", "daily_sum": 0
  }
}
```

Ответ:

```
{
  "decision_id": "uuid-...",
  "allowed": false,
  "decision": "REJECT",
  "reasons": [{"rule_id": "AML_102", "reason_code": "AML_COUNTRY_BLOCKED", "owner": "com
pliance"}],
  "risk_score": 0.95,
  "rules_evaluated": 2,
  "rules_matched": 1
}
```

5.4 GET /decision/rules + PATCH /decision/rules/{rule_id}

GET возвращает массив всех правил. PATCH позволяет обновить active и/или condition_params без деплоя.

Пример PATCH — изменить дневной лимит:

```
PATCH /decision/rules/LIMIT_DAILY
{
  "condition_params": {
    "fields": ["daily_sum", "amount"],
    "operator": "SUM_GT",
    "threshold": 5000000
  }
}
```

После PATCH следующий вызов /decision/evaluate немедленно использует новый порог. Перезапуск сервисов не требуется.

5.5 GET /decision/audit/{decision_id}

Возвращает полную запись из decision_audit по decision_id:

```
{
  "decision_id": "uuid-...",
  "final_decision": "REJECT",
  "risk_score": 0.95,
  "correlation_id": "corr-xyz",
```

```
  "created_at": "2026-03-26T12:00:00Z",
  "transfer_context": {"amount": 50000, "country": "IR", ...},
  "rules_checked": [
    {"rule_id": "LIMIT_DAILY", "matched": false, "action": "REJECT", "priority": 1},
    {"rule_id": "AML_102", "matched": true, "action": "REJECT", "priority": 2}
  ],
  "rules_matched": [{"rule_id": "AML_102", "reason_code": "AML_COUNTRY_BLOCKED", "owner"
: "compliance"}]
}
```

5.6 POST /ledger/posting

Запрос:

```
{
  "transfer_id": "uuid-...",
  "amount": 50000,
  "currency": "KZT",
  "correlation_id": "corr-xyz"
}
```

Если posting с данным transfer_id уже существует — возвращает существующую запись (идемпотентность). Уникальность обеспечивается на уровне БД (UNIQUE на transfer_id в ledger_postings).

6. Логика принятия решений

6.1 Правила в seed-данных

При первом запуске decision-hub выполняет Alembic-миграцию `002_seed_rules.py`, которая идемпотентно вставляет три начальных правила:

rule_id	version	priority	owner	Условие	action	risk_score
LIMIT_DAILY	1.0	1	fincontrol	daily_sum + amount > 10 000 000	REJECT	0.70
AML_102	2.1	2	compliance	country in {IR, KP, CU, SY}	REJECT	0.95
FRAUD_017	1.3	3	risk	device_trust=LOW AND amount > 200 000	REJECT	0.91

condition_params для каждого правила:

LIMIT_DAILY:

```
{"fields": ["daily_sum", "amount"], "operator": "SUM_GT", "threshold": 10000000}
```

AML_102:

```
{"field": "country", "blocked_values": ["IR", "KP", "CU", "SY"]}
```

FRAUD_017:

```
{"conditions": [{"field": "device_trust", "eq": "LOW"}, {"field": "amount", "gt": 200000
}]}
```

6.2 Стратегия вычисления

Движок (`app/engine/rule_engine.py`) реализует следующую логику:

- Правила сортируются по priority ASC перед вычислением
- Вычисляются только правила с active=True
- Для каждого правила вызывается evaluate_condition(rule, context) → bool
- Если правило сработало и action=REJECT → немедленный выход, decision=REJECT
- Если правило сработало и action=CHALLENGE → добавляется в список причин, вычисление продолжается
- После прохода всех правил: если есть CHALLENGE-причины → decision=CHALLENGE, allowed=True
- Если ни одно правило не сработало → decision=APPROVE, allowed=True, risk_score=null
- risk_score = максимальный из _RULE_RISK_SCORES для сработавших правил

Важно:

 CHALLENGE означает «разрешить, но пометить для ручной проверки». allowed=True при CHALLENGE — намеренное решение: операция проходит, но получает флаг для compliance-команды.

6.3 Типы условий (condition_type)

condition_type	Описание	Обязательные поля в condition_params
THRESHOLD	Числовое сравнение. Поддерживает суммирование нескольких полей.	fields: list[str], operator: SUM_GT, threshold: float
BLOCKLIST	Поле должно входить в список запрещённых значений.	field: str, blocked_values: list[str]
COMPOSITE	AND-комбинация нескольких sub-условий. Все должны выполняться.	conditions: [{field, eq?}, {field, gt?}, {field, lt?}]

7. Нефункциональные требования

7.1 Идемпотентность

Идемпотентность — свойство системы, при котором повторный идентичный запрос не порождает новые сайд-эффекты. В данной системе реализуется на двух уровнях:

Уровень 1: payment-service (Idempotency-Key)

- Каждый POST /p2p/transfer обязан содержать заголовок Idempotency-Key (UUID4)
- При первом запросе: создать transfer, выполнить flow, сохранить response_snapshot в payment_idempotency
- При повторном запросе с тем же ключом: вернуть response_snapshot из кэша (HTTP 200, idempotent: true)
- Клиент не получает 409 — он получает ровно тот же ответ, что и при первом запросе
- Идемпотентность сохраняется для всех конечных состояний: POSTED, REJECTED, FAILED

Уровень 2: ledger-mock (UNIQUE на transfer_id)

- Таблица ledger_postings имеет UNIQUE constraint на transfer_id
- Если POST /ledger/posting вызван с уже существующим transfer_id — вернуть существующую запись
- Это защита от двойного списания даже если payment-service вызовет ledger дважды

7.2 Структурированное логирование

Каждая строка лога во всех сервисах — JSON-объект. Реализуется через `shared/logging.py` (JSONFormatter + get_logger).

Обязательные поля в каждой лог-строке:

Поле	Тип	Описание
timestamp	ISO 8601 UTC	Время события
service	string	Имя сервиса: payment-service, decision-hub, ...
level	string	INFO | WARNING | ERROR
correlation_id	string | null	X-Correlation-Id для трассировки
event	string	Машиночитаемый тип события: transfer_created, status_transition, ...

Пример лог-строки (status_transition):

```
{
  "timestamp": "2026-03-26T12:00:00.000Z",
  "service": "payment-service",
  "level": "INFO",
  "correlation_id": "corr-abc-123",
  "event": "status_transition",
  "transfer_id": "uuid-...",
```

```
  "from_status": "DECIDED",
  "to_status": "POSTED"
}
```

7.3 Propagation X-Correlation-Id

Каждый запрос в системе должен нести X-Correlation-Id для сквозной трассировки:

Шаг	Действие
1. Клиент → api-gateway	Клиент может передать X-Correlation-Id. Если нет — api-gateway генерирует UUID4.
2. api-gateway → payment-service	Заголовок пробрасывается как есть.
3. payment-service → decision-hub	Передаётся в поле correlation_id тела запроса POST /decision/evaluate.
4. payment-service → ledger-mock	Передаётся в заголовке X-Correlation-Id.
5. Все сервисы	Сохраняют correlation_id в каждой лог-строке и в записях БД (transfer, audit).

Реализуется через `ContextVar` в `shared/correlation.py`: middleware сохраняет ID в контекст, логгер читает его автоматически при каждом вызове.

8. Статус-машина перевода

payment-service является единственным владельцем статуса перевода. Статус — явный, хранится в payment_transfers.status. Нет неявных переходов. Нет статуса «в процессе».

Допустимые переходы:

```
NEW → DECIDED → POSTED       (happy path: decision approved, ledger success)
NEW → DECIDED → FAILED       (decision approved, ledger failed)
NEW → REJECTED               (decision hub returned REJECT)
NEW → FAILED                 (decision hub unreachable)
```

Из статуса	В статус	Условие	Деньги moved?
NEW	DECIDED	Decision Hub вернул APPROVE или CHALLENGE	Нет
NEW	REJECTED	Decision Hub вернул REJECT	Нет
NEW	FAILED	Decision Hub недоступен (сетевая ошибка/timeout)	Нет
DECIDED	POSTED	Ledger-mock вернул успешный ответ	ДА
DECIDED	FAILED	Ledger-mock вернул ошибку или timeout	Нет

Значение статусов:

Статус	Смысл	Деньги переведены?	Финальный?
NEW	Запись перевода создана, ничего не вычислено	Нет	Нет
DECIDED	Decision Hub разрешил, ожидается проводка	Нет	Нет
POSTED	Ledger подтвердил проводку	ДА	Да
REJECTED	Правило заблокировало перевод	Нет	Да
FAILED	Инфраструктурный сбой (любой из вызовов)	Нет	Да

 DECIDED ≠ POSTED. Это два разных факта: решение принято и деньги переведены. В случае сбоя ledger статус DECIDED → FAILED. Деньги не двигались. Это намеренная архитектурная точность.

9. Shared-библиотека

Каталог `shared/` содержит два модуля, используемые всеми четырьмя сервисами. При сборке Docker-образов `shared/` копируется в `/service/shared/` и добавляется в PYTHONPATH.

shared/logging.py

Предоставляет `get_logger(service_name: str)` — возвращает стандартный Python logger с JSONFormatter. Каждый сервис вызывает один раз при инициализации.

Требования к JSONFormatter:

- Поля timestamp (UTC ISO 8601), service, level, correlation_id, event обязательны в каждой строке
- correlation_id читается из ContextVar (shared/correlation.py) — не нужно передавать явно
- Дополнительные поля из extra={} добавляются в тот же JSON-объект
- Если record.getMessage() не пустой и event не задан явно — использовать как event

shared/correlation.py

Управляет X-Correlation-Id через `contextvars.ContextVar`.

Требования:

- CorrelationMiddleware(BaseHTTPMiddleware) — при каждом запросе читает X-Correlation-Id из заголовка и сохраняет в ContextVar
- Параметр generate_if_missing: bool — если True (decision-hub), генерирует UUID если заголовок отсутствует; если False (payment-service), оставляет None
- get_correlation_id() → str | None — возвращает текущее значение из ContextVar
- set_correlation_id(value: str) — устанавливает значение вручную
- api-gateway устанавливает с generate_if_missing=True
- payment-service устанавливает с generate_if_missing=False (gateway уже обеспечил наличие)

10. Конфигурация и деплой

10.1 Переменные окружения

Переменная	Используется в	Значение по умолчанию	Описание
DATABASE_URL	payment-service, decision-hub, ledger-mock	postgresql+asyncpg://user:pass@postgres:5432/bankdb	Строка подключения к PostgreSQL (asyncpg driver)
PAYMENT_SERVICE_URL	api-gateway	http://payment-service:8001	Базовый URL payment-service
DECISION_HUB_URL	payment-service	http://decision-hub:8002	Базовый URL decision-hub
LEDGER_URL	payment-service	http://ledger-mock:8003	Базовый URL ledger-mock
LOG_LEVEL	все сервисы	INFO	Уровень логирования: DEBUG | INFO | WARNING | ERROR

Все переменные хранятся в `.env` (не коммитится) на основе `.env.example`.

10.2 Docker Compose

Порядок запуска сервисов (зависимости через healthcheck):

```
postgres (healthcheck: pg_isready)
  ├── decision-hub (depends_on: postgres)
  ├── ledger-mock  (depends_on: postgres)
  └── payment-service (depends_on: decision-hub, ledger-mock)
        └── api-gateway (depends_on: payment-service)
```

Требования к Dockerfile каждого сервиса:

- Build context: корень репозитория (не подкаталог сервиса)
- COPY shared/ /service/shared/ — до копирования кода сервиса
- COPY services/{service-name}/ . — код сервиса
- ENV PYTHONPATH=/service — чтобы from shared.logging import ... работало
- CMD: alembic upgrade head && uvicorn app.main:app --host 0.0.0.0 --port {port}
- Healthcheck: curl -f http://localhost:{port}/health

10.3 Alembic-миграции

Три сервиса (payment-service, decision-hub, ledger-mock) используют Alembic. Каждый мигрирует только свои таблицы.

Требования к env.py (async):

- Использовать async_engine_from_config + connection.run_sync(do_run_migrations)
- DATABASE_URL читать из os.environ['DATABASE_URL']
- poolclass=pool.NullPool — не держать пул во время миграции

- target_metadata = Base.metadata — импортировать модели явно

Структура миграций decision-hub:

```
001_create_tables.py  — создать decision_rules, decision_audit
002_seed_rules.py     — идемпотентно вставить LIMIT_DAILY, AML_102, FRAUD_017
```

 002_seed_rules.py должен быть идемпотентным: перед INSERT проверять существование по rule_id. Повторный запуск миграции не должен падать с ошибкой дублирования.

11. Структура репозитория

Ниже приведена полная структура файлов, которые должны быть созданы. Каждый файл несёт конкретную ответственность.

```
/
├── docker-compose.yml              # оркестрация 4 сервисов + postgres
├── .env.example                    # шаблон переменных окружения
│
├── shared/                         # общий код для всех сервисов
│   ├── __init__.py
│   ├── logging.py                  # JSONFormatter + get_logger()
│   └── correlation.py              # CorrelationMiddleware + ContextVar
│
├── contracts/                      # OpenAPI 3.1 (источник правды об API)
│   ├── api-gateway.openapi.yaml
│   ├── payment-service.openapi.yaml
│   ├── decision-hub.openapi.yaml
│   └── ledger-mock.openapi.yaml
│
├── docs/                           # документация
│   ├── sequence-legacy.puml        # PlantUML: AS-IS flow
│   ├── sequence-v2.puml            # PlantUML: TO-BE flow
│   ├── state-machine-transfer.md   # статус-машина перевода
│   └── decision-hub-rules.md       # каталог правил
│
├── scripts/
│   └── demo.sh                     # 4 демо-сценария (curl)
│
└── services/
    ├── api-gateway/
    │   ├── Dockerfile
    │   ├── requirements.txt
    │   └── app/
    │       ├── main.py             # proxy + correlation injection
    │       └── __init__.py
    │
    ├── payment-service/
    │   ├── Dockerfile
    │   ├── requirements.txt
    │   ├── alembic.ini
    │   └── app/
    │       ├── main.py
    │       ├── models.py           # PaymentTransfer, PaymentIdempotency
    │       ├── db/
    │       │   ├── database.py     # async engine + get_db
    │       │   └── migrations/
    │       │       ├── env.py      # async Alembic setup
    │       │       └── versions/
    │       │           └── 001_create_tables.py
    │       ├── routes/
    │       │   ├── transfer.py     # TO-BE 8-step flow
```

```
│       │   └── transfer_legacy.py  # AS-IS антипаттерн
│       └── services/
│           ├── decision_client.py  # httpx client для decision-hub
│           └── ledger_client.py    # httpx client для ledger-mock
│
├── decision-hub/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── alembic.ini
│   └── app/
│       ├── main.py
│       ├── models.py           # DecisionRule, DecisionAudit
│       ├── db/
│       │   ├── database.py
│       │   └── migrations/
│       │       ├── env.py
│       │       └── versions/
│       │           ├── 001_create_tables.py
│       │           └── 002_seed_rules.py
│       ├── engine/
│       │   └── rule_engine.py  # evaluate_condition + run_evaluation
│       └── routes/
│           ├── evaluate.py     # POST /decision/evaluate
│           └── admin.py        # GET/PATCH rules, GET audit
│
└── ledger-mock/
    ├── Dockerfile
    ├── requirements.txt
    ├── alembic.ini
    └── app/
        ├── main.py             # POST /ledger/posting + fail modes
        ├── models.py           # LedgerPosting
        └── db/
            ├── database.py
            └── migrations/
                ├── env.py
                └── versions/
                    └── 001_create_tables.py
```

12. Демо-сценарии

Система должна поддерживать 4 демо-сценария, запускаемых через `./scripts/demo.sh [A|B|C|D]` или все вместе без аргументов.

Сценарий A — Объяснимость решения

Вопрос: почему транзакция отклонена?
Демонстрирует разницу между AS-IS и TO-BE ответами при одинаковом входном запросе (страна: IR — Иран, в списке AML).

```
TO-BE: POST /api/p2p/transfer
  Ответ: {status: REJECTED, decision: {reasons: [{rule_id: AML_102,
          reason_code: AML_COUNTRY_BLOCKED, owner: compliance}], ...}}

AS-IS: POST /api/p2p/transfer-legacy
  Ответ: {status: REJECTED, reason: AML_BLOCKED}   ← только это
```

После получения decision_id из TO-BE ответа: `GET /decision/audit/{decision_id}` возвращает полный snapshot: что проверялось, что сработало, кто владелец.

Сценарий B — Изменение правила без деплоя

Цель: снизить дневной лимит немедленно, без пересборки образов.

- Отправить перевод 500 000 → проходит (500K < 10M лимит)
- PATCH /decision/rules/LIMIT_DAILY c threshold: 100000
- Тот же перевод 500 000 → отклоняется (500K > 100K новый лимит)
- Восстановить оригинальный порог через ещё один PATCH

Ни один контейнер не перезапускается. Между шагами 1 и 3 — только один API-вызов.

Сценарий C — Идемпотентность

Цель: ретрай не создаёт второй перевод и вторую проводку.

```
# Первый запрос
POST /api/p2p/transfer  (Idempotency-Key: stable-key-001)
Ответ: {transfer_id: uuid-abc, status: POSTED, idempotent: false}

# Повторный запрос (симуляция сетевого ретрая)
POST /api/p2p/transfer  (Idempotency-Key: stable-key-001, те же данные)
Ответ: {transfer_id: uuid-abc, status: POSTED, idempotent: true}
```

Оба ответа содержат одинаковый transfer_id. В таблице ledger_postings — одна запись.

Сценарий D — Частичный сбой (DECIDED ≠ POSTED)

Цель: показать что decision и проводка — независимые факты.

```
# Шаг 1: принудительный сбой ledger
POST /api/p2p/transfer  (X-Fail-Mode: ERROR)
Ответ: {status: FAILED, decision: {decision: APPROVE, ...}}
```

```
  → Decision Hub одобрил. Ledger упал. Деньги НЕ двигались.

# Шаг 2: повторить без fail mode
POST /api/p2p/transfer  (новый Idempotency-Key)
Ответ: {status: POSTED, ...}
```

13. Антипаттерн AS-IS vs TO-BE

Файл `services/payment-service/app/routes/transfer_legacy.py` является намеренным антипаттерном. Он существует для образовательного сравнения и должен оставаться рабочим кодом.

Проблемы, которые демонстрирует transfer_legacy.py:

#	Проблема	Проявление в коде
1	Логика встроена в сервис	_AML_BLOCKED_COUNTRIES и _DAILY_LIMIT — константы внутри файла
2	Нет аудит-трейла	При отклонении не создаётся ни одна запись в БД
3	Нет версионирования	Версия правила существует только в git blame
4	Нет структуры в ответе	reason: AML_BLOCKED — строка, не объект
5	Нет владельца	Кто изменит список стран? Разработчик, не compliance
6	Дублирование логики	Каждый новый сервис копирует этот if-else
7	Сложное тестирование	Правило нельзя проверить без HTTP-запроса к сервису

Сравнительная таблица AS-IS vs TO-BE:

Возможность	AS-IS (legacy)	TO-BE (Decision Hub)
Где живёт логика решений?	Внутри сервиса (код)	decision-hub (данные + движок)
Почему отклонено?	Строка в ответе	rule_id + reason_code + owner
Кто владелец правила?	Никто / git blame	Поле owner в каждом правиле
Изменить порог	Код → PR → деплой (дни)	PATCH /decision/rules (секунды)
Аудит-трейл	Отсутствует	Таблица decision_audit, каждое решение
Версионирование правил	Git история	version + updated_at в decision_rules
Тестирование изменений	Деплой на staging	PATCH + один API-вызов
Добавить логику в новый сервис	Copy-paste (расходится)	Вызов decision-hub (единый источник)

14. Критерии приёмки

Система считается реализованной корректно, если все следующие условия выполнены:

14.1 Функциональные критерии

#	Критерий	Как проверить
F1	POST /api/p2p/transfer с country=IR возвращает REJECTED с reasons[0].rule_id=AML_102	curl из Сценария A
F2	POST /api/p2p/transfer-legacy с тем же запросом возвращает только {reason: AML_BLOCKED}	Сравнить ответы A
F3	GET /decision/audit/{decision_id} возвращает полный snapshot с rules_checked	Сценарий A, шаг 3
F4	PATCH /decision/rules/LIMIT_DAILY с threshold=100000 немедленно блокирует 500K перевод	Сценарий B
F5	Повторный запрос с тем же Idempotency-Key возвращает idempotent=true, один posting в БД	Сценарий C
F6	POST /api/p2p/transfer с X-Fail-Mode=ERROR возвращает status=FAILED, decision=APPROVE	Сценарий D
F7	Все 4 сервиса возвращают HTTP 200 на GET /health после docker compose up	curl healthchecks
F8	POST /p2p/transfer без Idempotency-Key заголовка → HTTP 400	Прямой вызов

14.2 Нефункциональные критерии

#	Критерий	Как проверить
NF 1	Каждая строка лога — валидный JSON с полями timestamp, service, level, correlation_id, event	docker compose logs | python3 -c "import sys,json; [json.loads(l) for l in sys.stdin]"
NF 2	X-Correlation-Id присутствует во всех лог-строках одного запроса	docker compose logs | grep {correlation_id}
NF 3	decision_audit содержит запись для каждого вызова /decision/evaluate (включая APPROVE)	SELECT COUNT(*) FROM decision_audit
NF 4	ledger_postings содержит ровно одну запись при повторных запросах с тем же transfer_id	SELECT COUNT(*) FROM ledger_postings WHERE transfer_id=...
NF 5	Alembic миграции выполняются без ошибок при каждом docker compose up --build	Проверить логи запуска

14.3 Команды для запуска и проверки

```
# Запуск системы
cp .env.example .env
docker compose up --build

# Проверка здоровья
curl http://localhost:8000/health
curl http://localhost:8001/health
curl http://localhost:8002/health
curl http://localhost:8003/health

# Запуск всех 4 демо-сценариев
./scripts/demo.sh

# Или по отдельности
./scripts/demo.sh A
./scripts/demo.sh B
./scripts/demo.sh C
./scripts/demo.sh D
```

Конец документа. *Версия 1.0 | 2026-03-26 | Decision Hub Sandbox*